Constrained Constructive Optimization (CCO) Algorithm Implementation

Mahmoud Elshekh Ali,
Supervised by Dr. Jakub Wagner

Politechnika Warszawska Pl. Politechniki 1, 00-661 Warszawa
https://github.com/Ionvak/CCO-implementation

Abstract. The CCO Algorithm provides a way to simulate coronary networks with high accuracy, allowing for efficient and less intrusive methods of patient care. This report aims to explain and implement the CCO algorithm in 2D, and to compare the implementation to other existing implementations such as those by Schreiner [1] and Jacquet [2].

1 Introduction

Diagnostic procedures for cardiovascular diseases have in recent decades benefited from the introduction of personalized computational modelling and monitoring methods, which provide non-invasive options that assist in patient care. One of these methods is the algorithm known as Constrained Constructive Optimization (CCO), which works by generating a binary tree under functional constraints, and which can be used to simulate arterial trees up to the pre-arteriolar level. The reason for this cutoff point is that real arterial networks switch from a binary topology to an arcade network at the arteriolar level. At the edge of the arteriolar level, all terminal segments are assumed to perfuse an equal area of tissue, referred to as micro-circulatory black-boxes, which are abstracted away.

The CCO algorithm works by iteratively building a binary tree that is optimized at multiple levels with respect to the total blood volume. This optimization simulates that of real arterial networks, where the blood volume is minimized to preserve blood, seeing that it is a limited and precious resource.

2 Model Description

2.1 Trees

A tree consists of any number of interconnected segments that begin at the edge of the perfusion area and repeatedly bifurcate before terminating at the black-box micro-circulatory areas that they perfuse (see Fig. 1 for an example).

Each tree is described by the physical and structural parameters provided in table 1.

Due to the binary mode of branching, the total number of segments for any tree can be calculated using:

$$N_{tot} = 2 \cdot N_{term} - 1 \tag{1}$$

Table 1: Parameters describing each tree.

Parameter	Description	Symbol
Viscosity	Fluid viscosity of blood within the tree	μ
Bifurcation Exponent	Controls the bifurcation behavior within the tree	γ
Perfusion Pressure	Pressure at the proximal end of the root segment	P_{perf}
Perfusion Flow	Flow at the proximal end of the root segment	Q_{perf}
Terminal Pressure	Pressure at the distal end of each terminal segment	P_{term}
Perfusion Radius	Radius of the perfusion area	r_{perf}
Terminal Count	Number of terminal segments within the tree	N_{term}
Toss Count	Number of failed toss trials before reducing d_{thresh}	N_{toss}

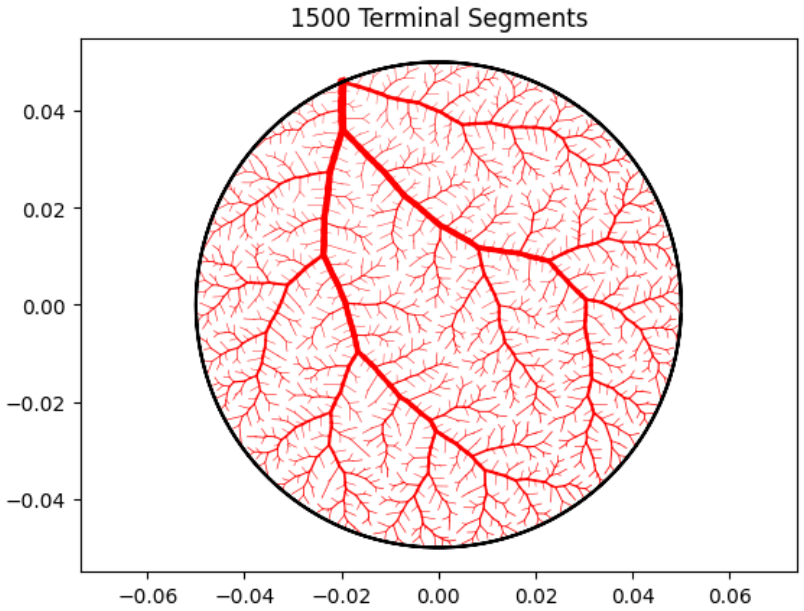


Fig. 1: An example of a fully constructed tree containing 1500 terminal segments.

2.2 The Perfusion Area

A finite three-dimensional or two-dimensional space of any shape can be used to represent the tissue region to be perfused. For simplicity, the perfusion area is assumed to be a two-dimensional geometric circle of a given radius r_{perf}, referred to as the perfusion radius. The algorithm works to ensure that this space is perfused as evenly as possible, while still minimizing the total volume of the tree and satisfying all the model requirements.

2.3 Segments

Segments within any tree are assumed to be simple geometric cylinders of a given length and radius. The segment end facing towards the root is referred to as the proximal end of the segment, while the end facing away from the root is referred to as the distal end. Each segment is fed by one parent segment and feeds two daughter segments. Each segment is represented using its distal end and that of its parent segment (the only exception is the root segment, which has no parent). The location, orientation, and length of each segment is defined by the Cartesian coordinates $x(i)$, $y(i)$ of its proximal and distal ends. Additionally, each segment is described using the following parameters:

Table 2: Parameters describing each segment.

Parameter	Description	Symbol
Index	An integer used to uniquely identify the segment	$\mathbf{i} \in \mathbb{Z}$
Parent	The index of the parent of the segment **i**	B_i
Left Child	The index of the left daughter of the segment **i**	D_i^l
Right Child	The index of the right daughter of the segment **i**	D_i^r

Each tree starts at a root segment (called *iroot*) for which there is no parent segment ($B_{iroot} = NULL$), and ends in N_{term} many terminal segments, for which there are no daughter segments ($D_i^l = D_i^r = NULL$). All terminal segments are assumed to perfuse an equal area.

2.4 Model Requirements

The target function to be optimized is the total volume within the tree, defined as:

$$T = \Sigma_i l(i)\pi r(i)^2 \tag{2}$$

where the length of the segment $l(i)$ is simply the Cartesian distance between the proximal and distal ends of the segment, and the radius is determined by the rescaling process.

The pressure drop along the path from the root to any of the terminal segments must be the same and must be equal to $P_{perf} - P_{term}$.

The flow Q_i through a segment is proportional to the number of terminal segments downstream from it:

$$Q_i = NDIST_i \cdot Q_{term} \tag{3}$$

$NDIST_i = 1$ for terminal all segments by definition.

Poiseuilles' law and Hagen-Poiseuilles' law describe the behavior of blood flow within any segment in the tree and provide a relation between the flow, resistance, and pressure drop in any segment:

$$R = \frac{8\mu l}{\pi r^4} \tag{4}$$

$$\Delta P = RQ = \frac{8\mu l Q}{\pi r^4} \tag{5}$$

The rescaling process involves calculating all the reduced hydrodynamic resistance values defined as $R_i^* = R_i r_i^4$ for all segments within the tree, which is used together with equations (4) and (5) and the bifurcation ratios to calculate the appropriate radii values for all segments. The following equation is used to calculate the reduced hydrodynamic resistance of a segment, including its left and right subtrees.

$$R^*_{sub,i} = \frac{8\mu l_i}{\pi} + (\frac{(r_{left,i}/r_i)^4}{R^*_{left,i}} + \frac{(r_{right,i}/r_i)^4}{R^*_{right,i}})^{-1} \tag{6}$$

The following equations are used to calculate the radii ratios between a segments' children, between a segment and its children, and to calculate the radius of the root segment (k represents the parent segment):

$$\frac{r_i}{r_j} = (\frac{Q_i R^*_{sub,i}}{Q_j R^*_{sub,j}})^{\frac{1}{4}} \tag{7}$$

$$\beta^i_k = (1 + (\frac{r_i}{r_j})^{-\gamma})^{-\frac{1}{\gamma}} \quad and \quad \beta^j_k = (1 + (\frac{r_i}{r_j})^{\gamma})^{-\frac{1}{\gamma}} \tag{8}$$

$$r_{iroot} = (R^*_{sub,iroot} \frac{Q_{perf}}{P_{perf} - P_{term}})^{\frac{1}{4}} \tag{9}$$

Finally, we calculate the radii of the segments:

$$r_i = r_{iroot} \prod_{k=i}^{iroot} \beta_k \tag{10}$$

3 Algorithm Description

3.1 Initializing the tree

The algorithm begins by initializing the process variables used throughout the algorithm:

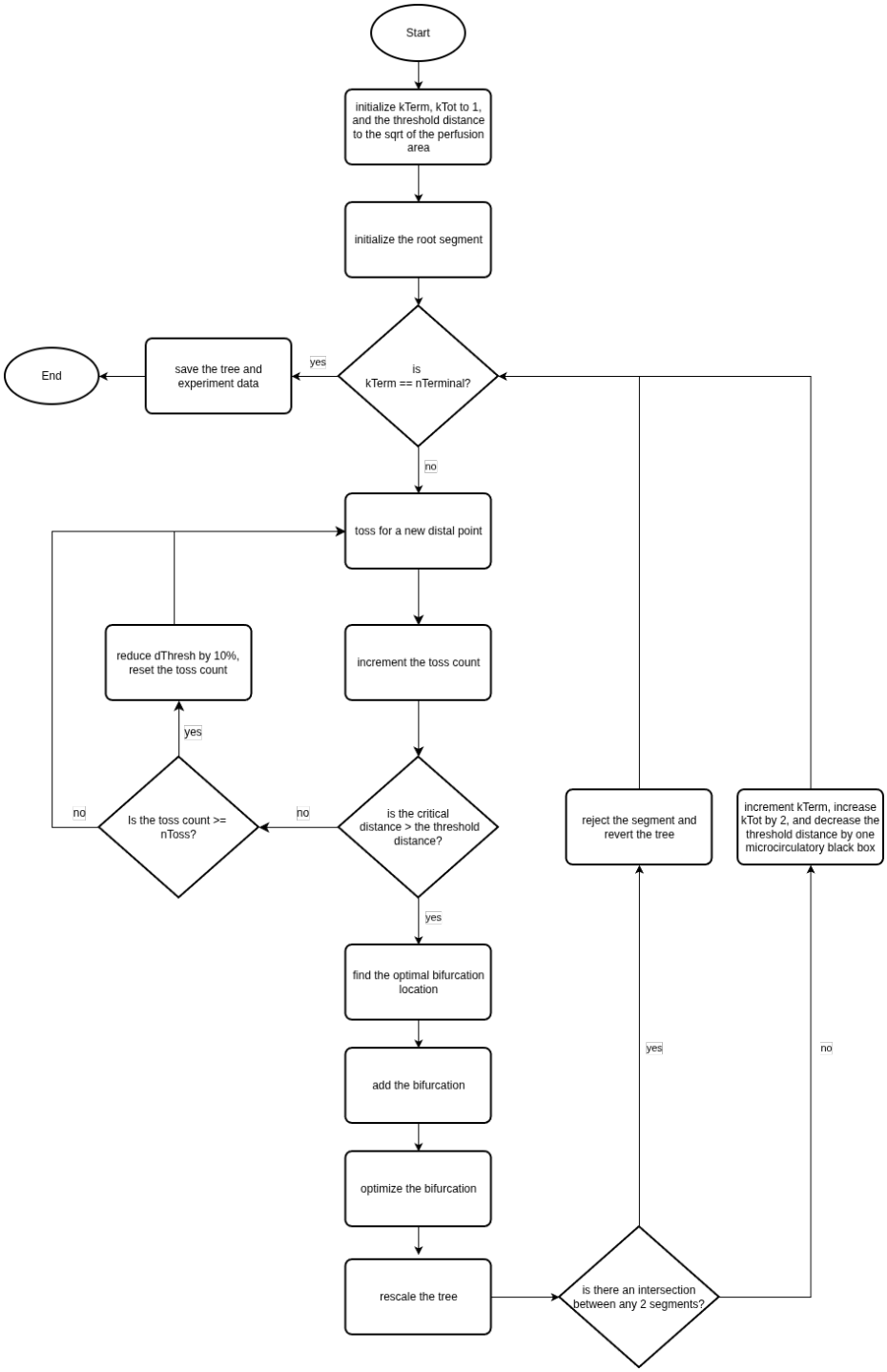


Fig. 2: A diagram showcasing the CCO algorithm.

Table 3: Variables used throughout the CCO algorithm.

Variable	Description	Initial Value
K_{term}	Number of terminal segments in the tree	1
K_{tot}	Number of segments in the tree	1
d_{thresh}	Threshold distance	$\sqrt{\pi \cdot r_{perf}^2}$
i_{conn}	Index of the segment to bifrucate	NULL
i_{new}	Index of the segment to be added	NULL
i_{old}	$B_{i_{conn}}$ prior to bifurcation	NULL
i_{bif}	$B_{i_{conn}}$ after bifurcation	NULL

3.2 Building the tree

Building the tree consists of repeatedly adding terminal segments until the desired number of terminal segments is reached. The first segment to be added is the root segment, whose proximal end is at a random point along the edge of the perfusion area, and whose distal end is at a random point inside the perfusion area (see Fig. 3). The root segment represents the main feeding artery.

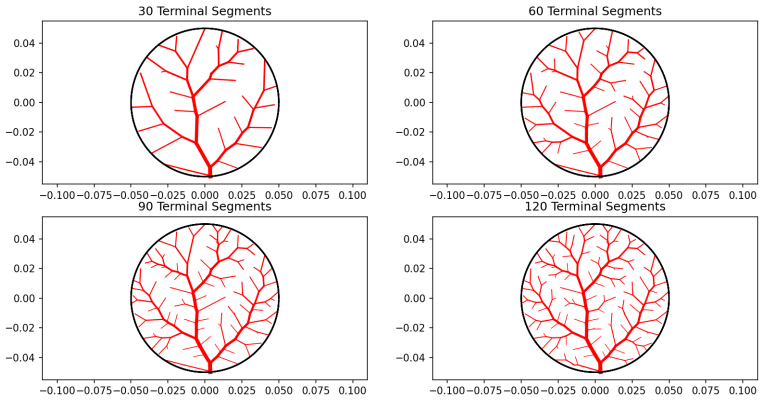


Fig. 3: An example of the tree construction process for a tree with 120 terminal segments.

Adding a new terminal segment starts with tossing for a new potential distal end for it. Tossing simply refers to the process of selecting a random point within the perfusion area with a uniform distribution. The point is accepted on the condition that the critical distance between the point and any of the existing segments in the tree exceed the threshold distance. The critical distance between

a point (x, y) and a segment j is defined as the orthogonal distance between them if the projection of the point (x, y) along the segment j is $0 \leq d_{ortho}(x, y, j) \leq 1$, otherwise it is the smaller of the distances between the point and either of the j's endpoints:

$$d_{ortho}(x, y, j) = \left| \begin{pmatrix} -y(B_j) + y(j) \\ x(B_j) - x(j) \end{pmatrix} \cdot \begin{pmatrix} x - x(j) \\ y - y(j) \end{pmatrix} \right| \cdot l(j)^{-1} \tag{11}$$

$$d_{proj}(x, y, j) = \begin{pmatrix} x(B_j) - x(j) \\ y(B_j) - y(j) \end{pmatrix} \cdot \begin{pmatrix} x - x(j) \\ y - y(j) \end{pmatrix} \cdot l(j)^{-2} \tag{12}$$

$$d_{end}(x, y, j) = min(\sqrt{(x - x(j))^2 + (y - y(j))^2}, \sqrt{(x - x(B_j))^2 + (y - y(B_j))^2}) \tag{13}$$

If a point fails to meet the critical distance criteria, it is rejected and the tossing process is repeated. If the tossing process is repeated N_{toss} times without success, the threshold distance is reduced by 10% and the loop count is reset before repeating the tossing process again; this is done until a valid point is found.

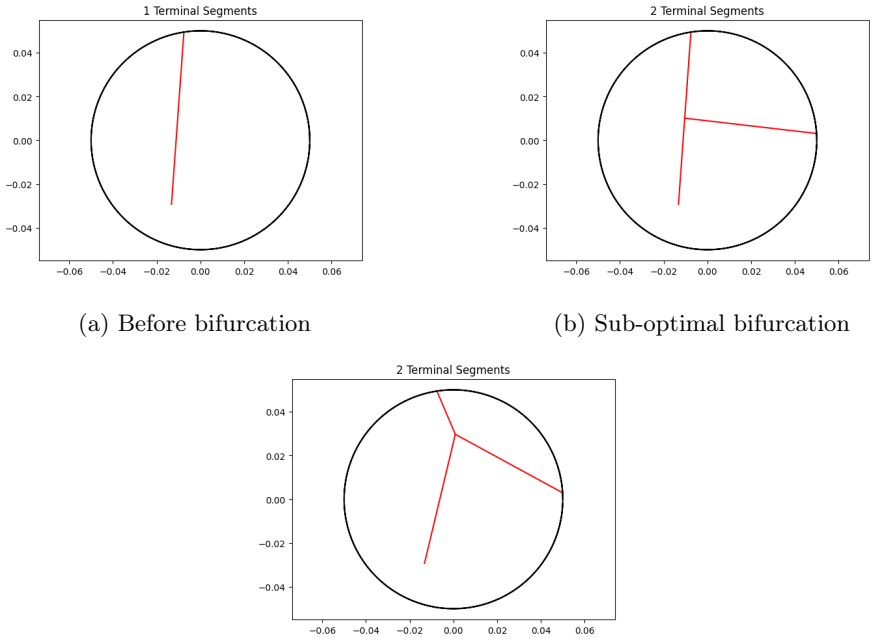
Adding bifurcations is a multi-step procedure for adding new terminal segments to the tree that begins after a distal point is found. Two segments are always added to the tree after each bifurcation, one of which is a terminal segment. A bifurcation is created by splitting an existing segment i_{conn} into two halves: the proximal half of i_{conn} is assigned to i_{bif}, and the distal half to i_{conn} itself; a new terminal segment i_{new}, whose distal end is the point found after tossing, and whose proximal point is the point at which the two halves are connected, is added to the tree. An example of the full bifurcation process is shown in Fig. 4.

The tree is checked for intersections whenever a bifurcation is added to ensure that the binary structure of the tree is not lost. This is done using the algorithm provided by Goldman [3]. If an intersection is detected, the changes to the tree are reverted, the bifurcation is rejected, and another bifurcation location is selected.

3.3 Optimizing bifurcations

Geometric optimization is used to ensure that all bifurcations are optimal with respect to the target function. The point at which i_{new}, i_{bif}, and i_{conn} are connected is shifted to the location minimizing the target function. This is accomplished using a Nelder-Mead optimizer, which iteratively finds the optimal location within the search space.

Selecting the optimal bifurcation location adds another layer of optimization to the tree. Whenever a distal point is found after tossing, the tree is searched for the optimal segment to bifurcate with respect to the target function. While it is possible to systematically search the whole tree for the optimal segment



(a) Before bifurcation

(b) Sub-optimal bifurcation

(c) optimal bifurcation

Fig. 4: An example of the segment addition process. The process begins with deciding where the bifurcation will occur, followed by creating a bifurcation and adding the new segment, after which the bifurcation is optimized with respect to the target function.

to bifurcate, the actual procedure only considers the 20 closest segments to the distal point found after tossing (assuming at least 20 segments are in the tree, otherwise all the segments are considered) to improve performance and remove redundant searches. Each segment from those considered is tested: the bifurcation is added and optimized, and the resulting value of the target function is stored. The segment yielding the lowest target function value is selected as the optimal candidate.

3.4 Rescaling the tree

The radii of the segments in the tree are calculated based on the physical parameters and the topological structure of the tree to ensure consistency with actual coronary networks. The segment radii must be recalculated and reset after any change to the topological structure of the tree. A multi-step, recursive procedure, referred to as rescaling, is used to find the appropriate radii values.

Rescaling begins at the terminal segments, where equation (6) is used to find the reduced hydrodynamic resistance. Since the terminal segments contain no

left or right subtrees, equation (6) is reduced to the first term only. Once the reduced hydrodynamic resistance of the terminal segments is found, it is used to recursively calculate the reduced hydrodynamic resistance of all segments up to the root. Knowing the reduced resistance, it is then possible to calculate the radii ratios between the children of each segment (except for the terminal segments since they have no children) using equation (7). These ratios are used in turn to find the radii ratios between each child of each segment and the segment itself using equations (8). Finally, the root radius is calculated using equation (9) and used with equation (10) to recursively find the radii of all the segments.

4 Results

After comparing trees with $N_{term} = 4000$ visually and analytically to those generated with similar physical parameters in other implementations, it seems that the trees obtained closely resemble those obtained by Schriener [1] and Jacquet [2]. Some differences can be observed, however those are plausibly explained by the difference in visualization tools, and the random nature of the tree generation process. See Fig. 5-7.

Table 4: Selected physical parameters (identical to those used by Schreiner [1]).

Parameter	Value	Units
μ	$3.6 \cdot 10^{-3}$	$Pa \cdot s$
P_{perf}	$1.33 \cdot 10^{4}$	Pa
Q_{perf}	$8.33 \cdot 10^{-6}$	m^3/s
P_{term} (for $N_{term} = 4000$)	$7.98 \cdot 10^{3}$	Pa
r_{perf}	0.05	m
γ	3.00	
N_{toss}	10	

One considerable difference between this and other implementations is the supporting circle stretching procedure, which is mentioned in other implementations but which was removed in this implementation. The method for controlling the spread of distal ends evenly across the perfusion area in this implementation is to instead use the whole perfusion area, while reducing the threshold distance after each segment addition. This proved to be more efficient while achieving very similar results. The tree generation process for a tree with $N_{term} = 4000$ in the case of Schreiner 1993 [1] took around 2 days, and that of Jacquet 2018 [2] took 33 hours; using the present implementation, it was possible to generate a tree with the same number of terminal segments in about an hour (the difference in computing power of the testing machines used is another important factor).

Comparing studies of segment diameters at different bifurcation levels, the results also appear largely consistent, with only minimal differences at lower bifurcation levels. See Fig. 8.

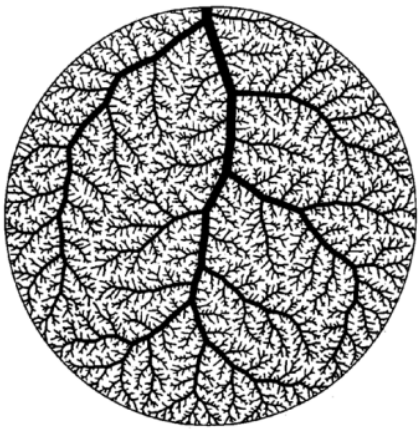


Fig. 5: Fig. 3 from Schreiner 1993 [1]. Tree with $N_{term} = 4000$.

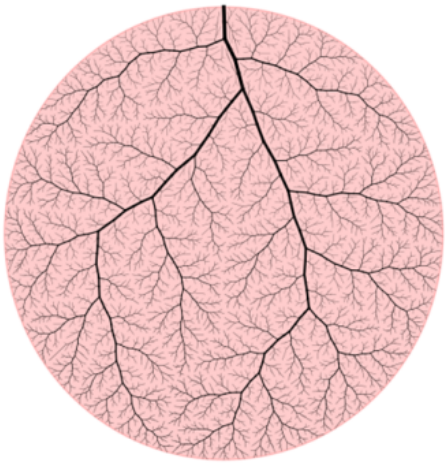


Fig. 6: Fig. 4 from Jacquet 2018 [2]. Tree with $N_{term} = 4000$.

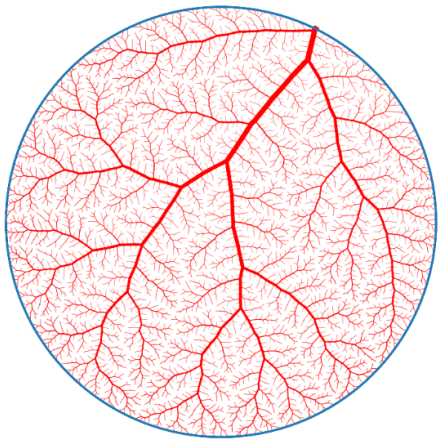


Fig. 7: Tree with $N_{term} = 4000$.

The pressure profile of various trees was tested and visualized to ensure that it meets the model requirements. A simple procedure was developed to test the pressure profile: the tree was recursively explored along each path from the root to the terminal segments. Along each path, a variable was initialized with the perfusion pressure, and was reduced every time a segment was encountered by the pressure drop of the segment until reaching a terminal segment. The pressure at the distal end of each terminal segment was verified to be equal to the terminal pressure with a tolerance of 0.01%. See Fig. 9.

References

1. Wolfgang Schreiner and Peter Franz Buxbaum. Computer-optimization of vascular trees. Biomedical Engineering, IEEE Transactions on, 40(5):482–491, 1993.
2. Clara Jaquet. Constrained Constructive Optimization Implementation: Reproducing Rudolph Karch Paper. Technical report, Paris-Est University, December 2018.
3. Goldman, R. (1990). Intersection of two lines in three-space. Graphics gems, 304.

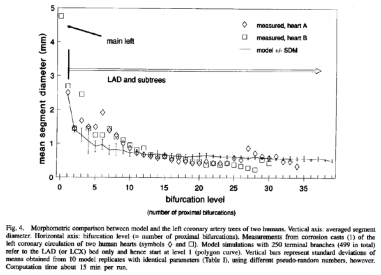
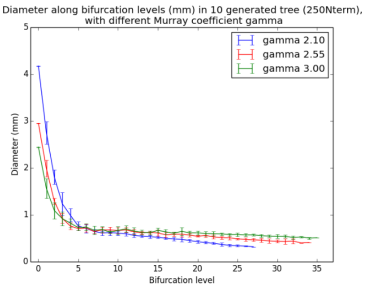
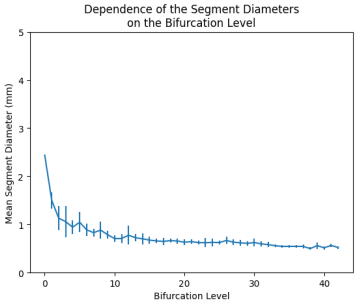


Fig. 4. Morphometric comparison between model and the left coronary artery trees of two humans. Vertical axis: averaged segment diameter. Horizontal axis: bifurcation level (= number of proximal bifurcations). Measurements from corrosion casts (◇) of the left coronary circulation of two human hearts (symbols ◇ and □). Model simulations with 250 terminal branches (499 in total) refer to the LAD (or LCX) bed only and hence start at level 1 (polygon curve). Vertical bars represent standard deviations of means obtained from 10 model replicates with identical parameters (Table I), using different pseudo-random numbers, however. Computation time about 15 min per run.

(a) From Schreiner [1].



(b) From Jacquet [2].



(c) Diameter study from 10 trees.

Fig. 8: Study of segment diameters at all bifurcation levels across 10 trees with $N_{term} = 250$ with the means and standard deviations of every level. Top left: from Schreiner [1], $\gamma = 2.55$ alongside real anatomical data. Top right: from Jacquet [2], $\gamma = 2.10, 2.55, 3.00$. Center: $\gamma = 3.00$.

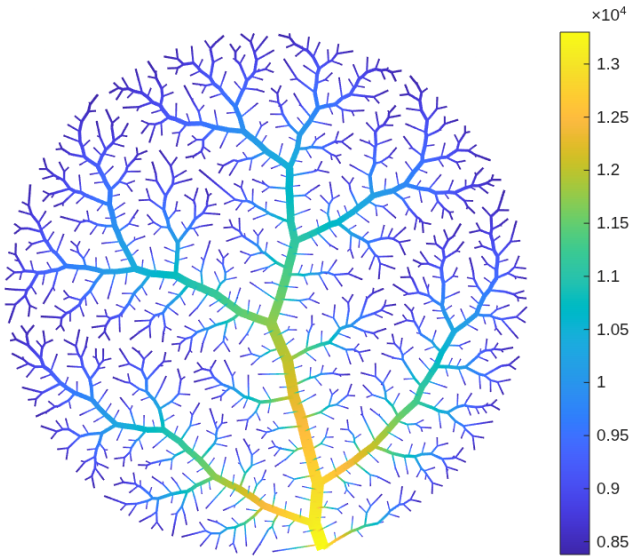


Fig. 9: The pressure profile in Pa of a tree with $N_{term} = 500$.